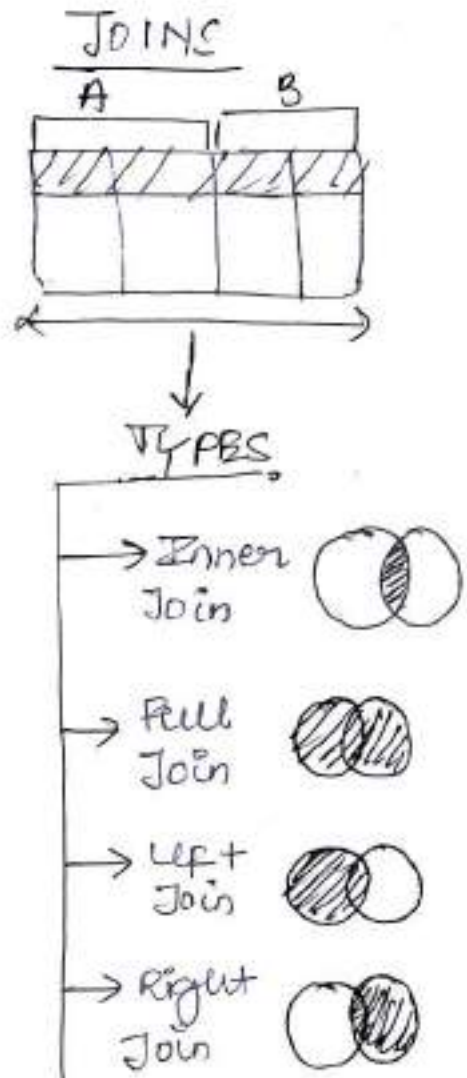
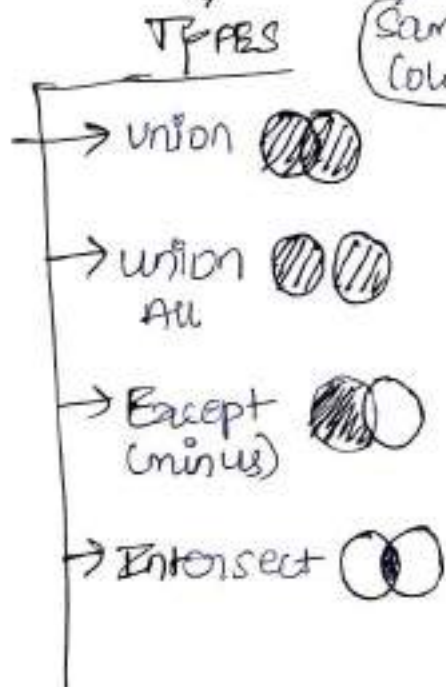
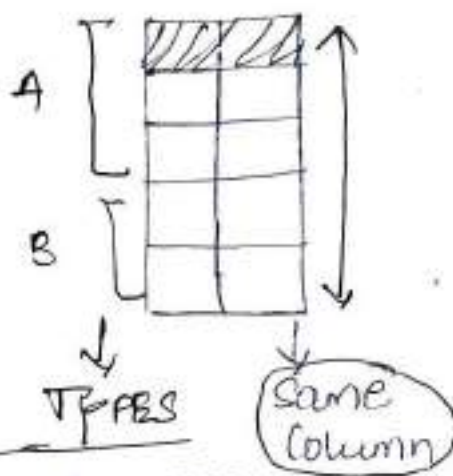
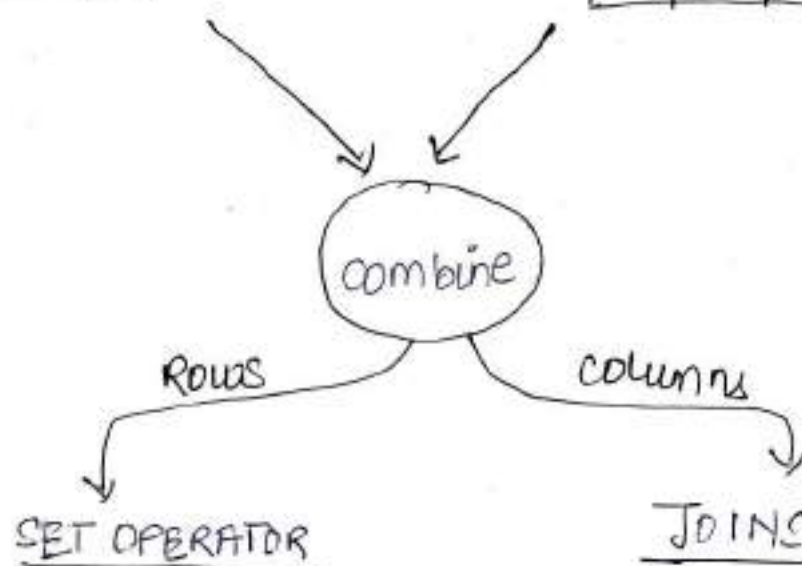


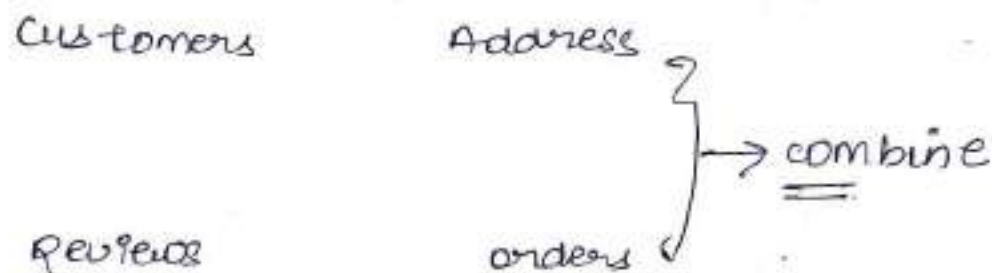


what is SQL join?

JOIN in SQL is used to combine rows from two or more tables based on a related column b/w them. Helps in Retrieving meaningful information from relational databases by connecting data stored in separate tables.

when to use SQL?

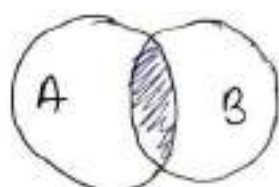
1) Recombine Data



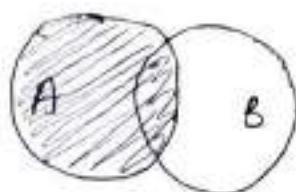
2) Data Enrichment "Greeting Extra Data"

3) check for Existence "Filtering"

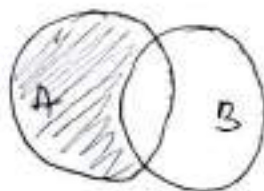
JOIN TYPES



Matching Data



All Data



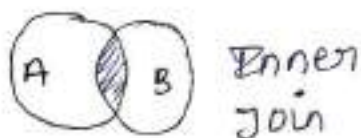
unMatching Data



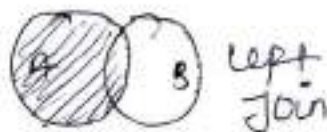
Basics



NO
Join



Inner
Join



Left
Join

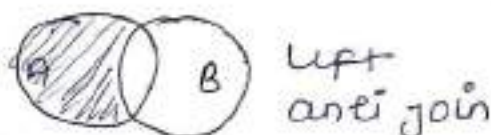


Right
Join



Full
Join

Advanced



Left
anti join



Right
anti join



Full
anti join



Cross Join

NO JOINS

Return data from Tables without combining them:



Two results NO Need to combine.

Syntax

```
SELECT *
```

```
FROM A
```

```
SELECT *
```

```
FROM B.
```

Ex

```
SELECT *
```

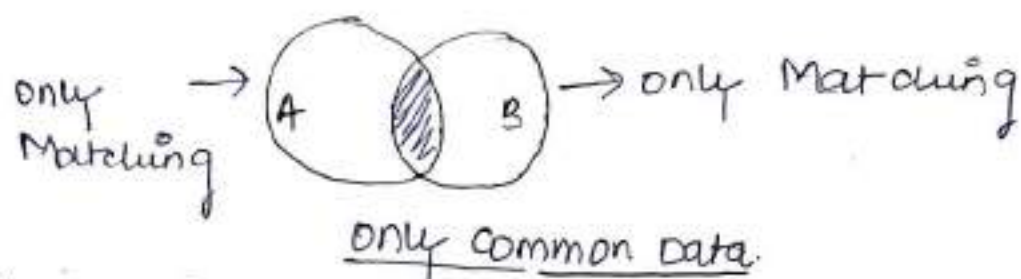
```
FROM customers;
```

```
SELECT *
```

```
FROM orders;
```


INNER JOIN

Returns only Matching Rows from Both Tables.



Syntax

```
SELECT *  
FROM A  
INNER [TYPE] JOIN B  
ON <condition>
```

Default inner join

$A.key = B.key$

- Order of Table Doesn't matter.

Ex:- Get all customer along with their order, but only for customer who have placed an order.

```
SELECT *  
FROM customers  
INNER JOIN orders  
ON id = customers_id
```

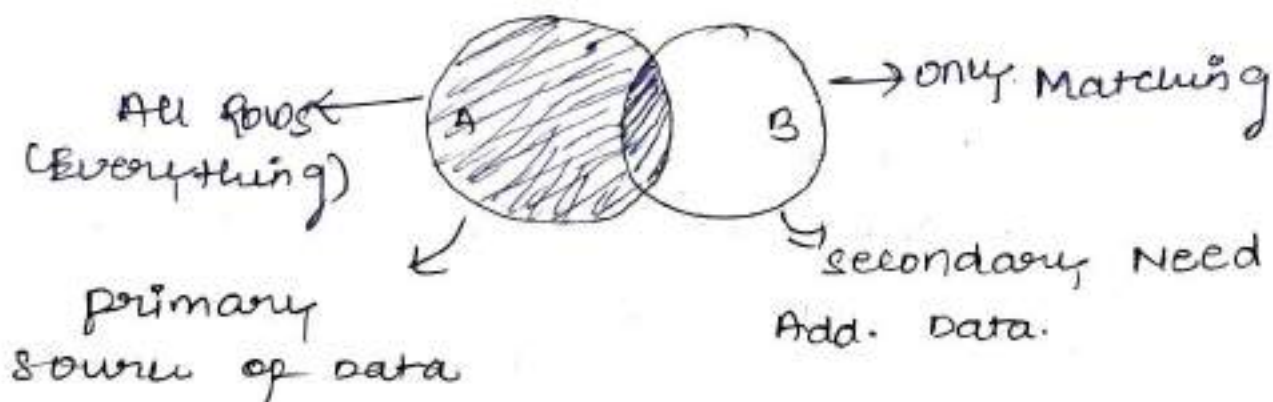
If we have same columns in both Table we use.

AS c	AS o
c.customer	o.id
c.id	o.sales

INNER JOIN uses poor → Recombine data
→ filtering / check Existence.

#. LEFT JOIN.

Returns all rows from left and only Matching from right.



```
SELECT *  
FROM A  
LEFT JOIN B  
ON A.KEY = B.KEY
```

Annotations:
← Left *
→ Right.

*order of Tables is important.

Ex: Get all customers along with their orders, including those without orders

```
SELECT  
C.id,  
C.first_name,  
O.order_id,  
O.sales
```

FROM customers AS C

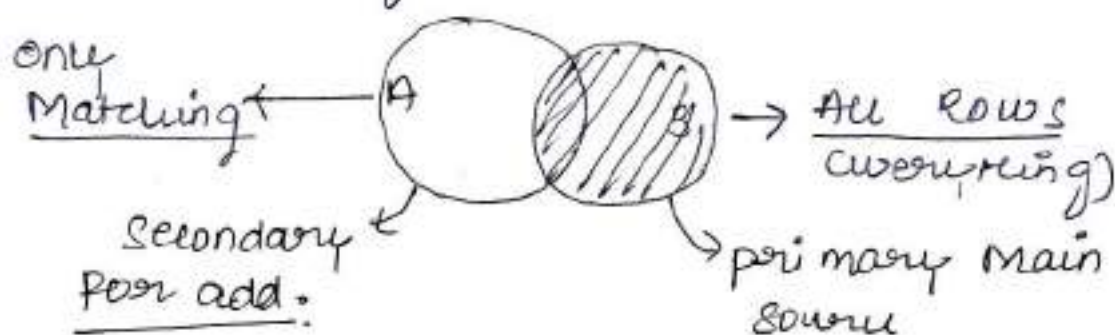
LEFT JOIN AS orders AS O

ON C.id = O.customer-id.

Left join use { Recombine Data
Data Enrichment.

RIGHT JOIN

Returns all rows from right and only
Matching from left.



```
SELECT *  
FROM A  
RIGHT JOIN B  
ON A.key = B.key
```

* ORDER of Tables is important.

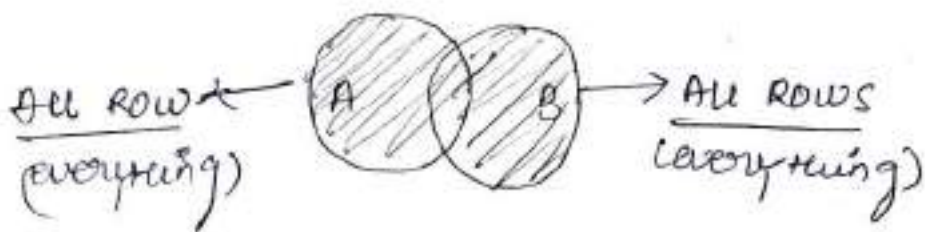
Q2: Get all customers along with their order including order without matching customers.

```
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales.
FROM customers AS c
RIGHT JOIN orders AS o
ON c.id = o.customer_id.
```

↓ Alternative

```
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM orders AS o
LEFT JOIN customers AS c
ON c.id = o.customer_id.
```


FULL JOIN



- Returns All ROWS FROM Both Tables.

```
SELECT *  
FROM A  
FULL JOIN B  
ON A.Key = B.Key
```

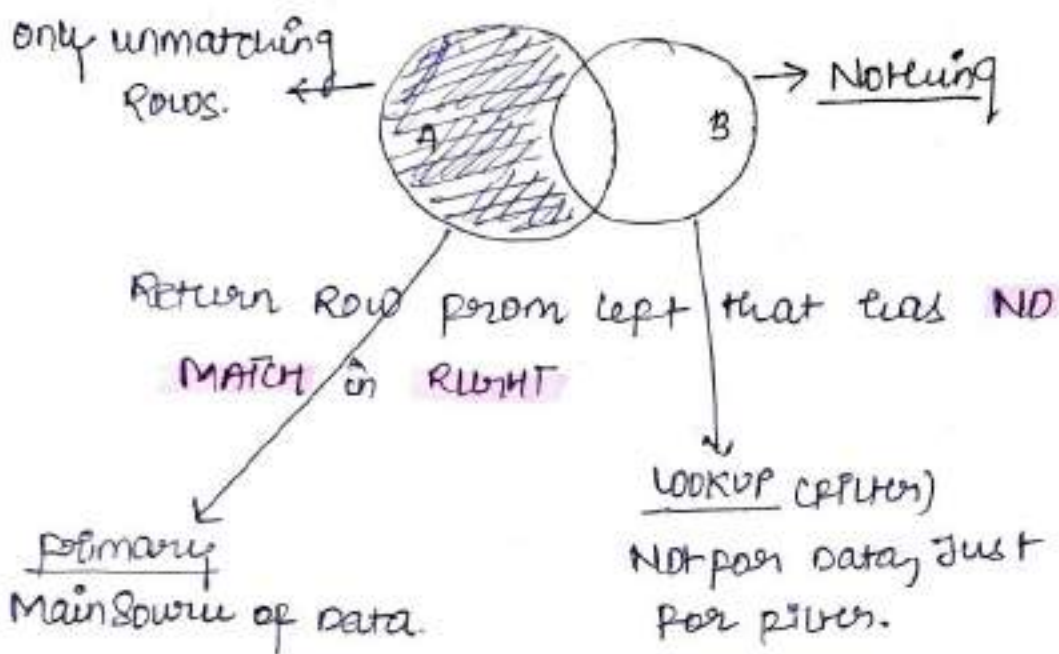
- order of Tables does not Matter

Ex: Get all customers and order, even if there is no match.

Full Join use where } Recombine
+ where } Filter check existence

ADVANCED JOIN.

Left Anti join



Syntax.

The order of Tables Is important.

```
SELECT *  
FROM A  
LEFT JOIN B  
ON A.key = B.key  
WHERE B.key IS NULL
```

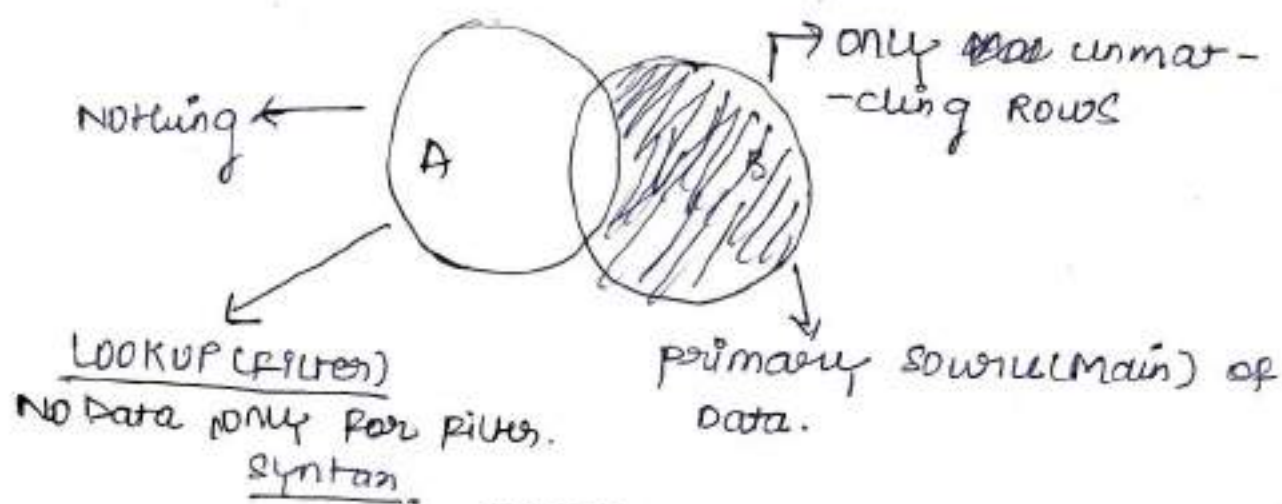
Ex:-

```
SELECT *  
FROM customers AS C  
LEFT JOIN orders AS O  
ON C.id = O.customer_id  
WHERE O.customer_id IS NULL.
```

LEFT anti join use to check existence.

RIGHT ANTI JOIN

Returns Rows From Right that has NO MATCH in left.



```
SELECT *  
FROM A  
RIGHT JOIN B  
ON A.Key = B.Key
```

WHERE A.Key IS NULL.

→ A is left/No Data

• Order of Tables must important.

Ex:

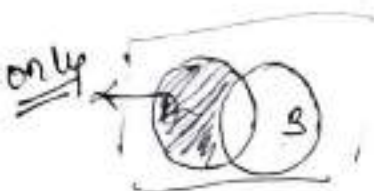
```
SELECT *  
FROM customers AS C  
RIGHT JOIN orders AS O  
ON C.id = O.customer_id  
WHERE C.id IS NULL.
```

↓
alternative way.
(LEFT JOIN)

LEFT JOIN (Method)

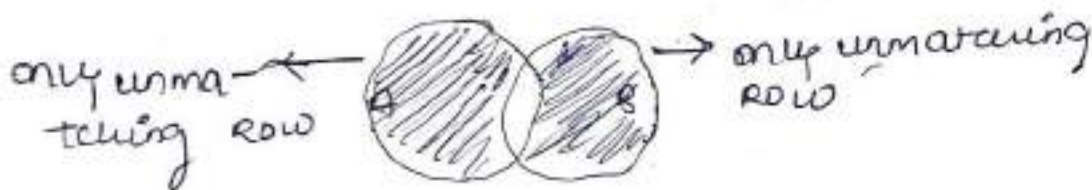


```
SELECT *  
  FROM orders AS D  
LEFT JOIN customers AS C  
ON D.customer-id = C.id  
WHERE customer-id IS NULL  
      C.id
```



FULL ANTI JOIN

- Returns only rows that don't match in either tables.



only unmatching row.

Syntax

```
SELECT *  
  FROM A  
FULL JOIN B  
ON A.key = B.key  
WHERE  
  B.key IS NULL  
  OR  
  A.key IS NULL
```

Note: Tables of order does not matter

Ex:-

[C.id Means order hai but without customer wala order]

SELECT *

FROM customers AS C

FULL JOIN orders AS O

ON C.id = O.customer-id

WHERE C.id IS NULL OR O.customer-id
IS NULL

Another ques.

[C.id IS NULL = order without customer.
O.customer-id IS NULL = cust. without order]

• Get all customer along with their orders,
but only for customers who have placed
an order without using INNER JOIN)

SELECT *

FROM customers AS C

LEFT JOIN orders AS O

ON C.id = O.customer-id

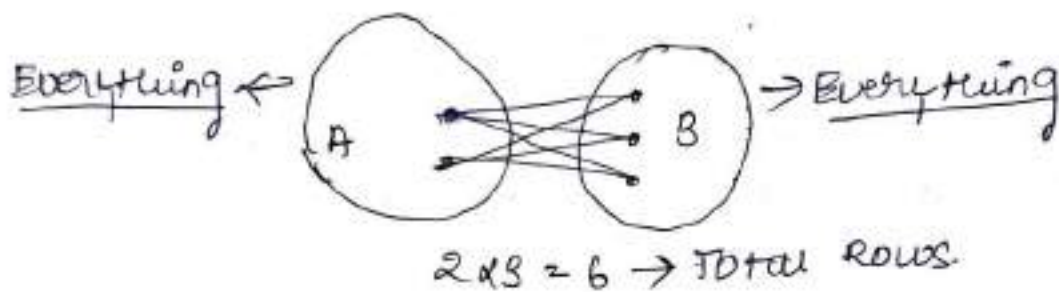
WHERE O.customer-id IS NOT NULL

✓
Sirf wahi row chahiye jaha
orders side me customer-id
NULL nahi hai

CROSS JOIN

Combines Every Row from Left with Every Row from Right.

All possible combinations - Cartesian join.



Syntax.

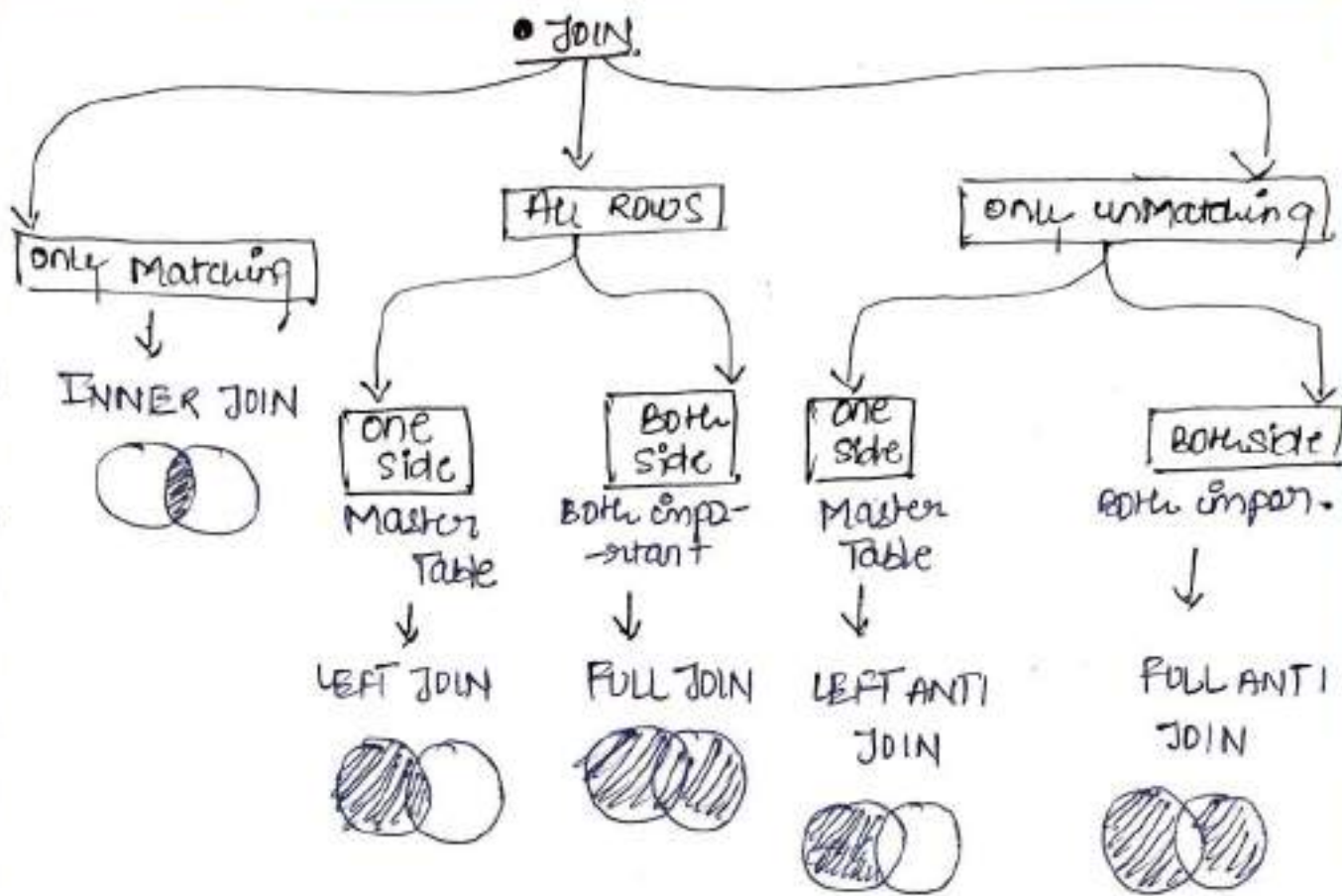
```
SELECT *  
FROM A  
CROSS JOIN B
```

$\swarrow$
No condition needed.

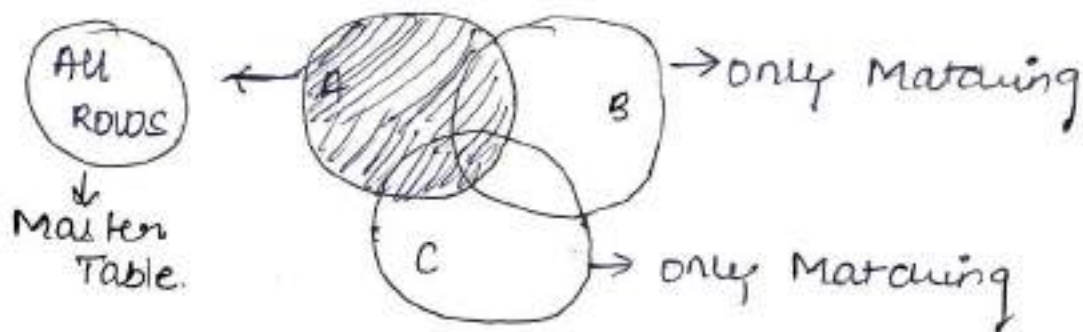
Rule: Order of Table does not Matter.

Eg: Generate all possible combination of customers and order

```
SELECT *  
FROM A  
CROSS JOIN B
```



Multiple Join Table advance.



```

SELECT *
FROM A
LEFT B ON ...
LEFT C ON ...
WHERE control what to keep
  
```

#overlapping data



Syntax.

```
SELECT +  
FROM A  
INNER B --  
INNER C ---
```

E.g.) using sales DB, retrieve list of all order, along with the related customer, product and employee details.

eg) For each order, display.

✓ order ID.

- customer name.
- product name.
- sales amount.
- product price.
- sales person's name.

SELECT

o.orderID,

o.sales * p.price AS sales Amount,

c.firstname AS customerfirstname,

c.lastname AS customerlastname,

p.product AS productName,

p.price AS product price,

e.firstname AS Employeefirstname,

e.lastname AS Employeelastname,

FROM sales.orders AS o

LEFT JOIN sales.customers AS c

ON o.customerID = c.customerID

LEFT JOIN sales.product AS p

ON o.productID = p.productID

LEFT JOIN sales.employees AS e

ON o.salespersonID = e.employeeID.